

lab8

YUE CHENGHAO

227154

Stored Procedures

- code

```
-- Q1: raise_amount(emp_id, raise_pct)
CREATE OR REPLACE PROCEDURE raise_amount (
    p_emp_id IN employees.employee_id%TYPE,
    p_raise IN NUMBER -- e.g. 0.10 = 10%
)
AS
BEGIN
    UPDATE employees
    SET salary = salary + (salary * p_raise)
    WHERE employee_id = p_emp_id;

    COMMIT;
END raise_amount;
/
SHOW ERRORS;

-- Q2: DisplayEmployeeInfo(emp_id)
CREATE OR REPLACE PROCEDURE DisplayEmployeeInfo (
    EmpNum IN employees.employee_id%TYPE
)
AS
    v_name employees.first_name%TYPE;
    v_job employees.job_id%TYPE;
    v_salary employees.salary%TYPE;
BEGIN
    SELECT first_name, job_id, salary
    INTO v_name, v_job, v_salary
    FROM employees
    WHERE employee_id = EmpNum;

    DBMS_OUTPUT.PUT_LINE('Employee Name: ' || v_name);
    DBMS_OUTPUT.PUT_LINE('Employee Position: ' || v_job);
```

```

        DBMS_OUTPUT.PUT_LINE('Salary: ' || v_salary);
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            DBMS_OUTPUT.PUT_LINE('No employee found for EmpNum = ' || EmpNum);
    END DisplayEmployeeInfo;
/
SHOW ERRORS;

-- Q3: insert_department (upsert: exists->update, not exists->insert)
CREATE OR REPLACE PROCEDURE insert_department (
    p_dept_no    IN NUMBER,
    p_dept_name  IN VARCHAR2
)
IS
BEGIN
    MERGE INTO departments d
    USING (SELECT p_dept_no AS dept_id, p_dept_name AS dept_name FROM dual)
    s
    ON (d.department_id = s.dept_id)
    WHEN MATCHED THEN
        UPDATE SET d.department_name = s.dept_name
    WHEN NOT MATCHED THEN
        INSERT (department_id, department_name)
        VALUES (s.dept_id, s.dept_name);

    COMMIT;
END insert_department;
/
SHOW ERRORS;

```

- result

	EMPLOYEE_ID	FIRST_NAME	JOB_ID	SALARY
1	100	Steven	AD_PRES	29040

	EMPLOYEE_ID	FIRST_NAME	JOB_ID	SALARY
1	100	Steven	AD_PRES	31944

	DEPARTMENT_ID	DEPARTMENT_NAME
1	300	Computer Science

Triggers

- code

```
-- Trigger Exercise 1: PURCHASE table + trigger
CREATE TABLE purchase (
    product VARCHAR2(16),
    quantity NUMBER,
    price    NUMBER(8,2),
    cost     NUMBER
);

CREATE OR REPLACE TRIGGER purchase_trig
BEFORE INSERT OR UPDATE ON purchase
FOR EACH ROW
BEGIN
    :NEW.cost := :NEW.quantity * :NEW.price;
END purchase_trig;
/
SHOW ERRORS;

-- Trigger Exercise 2: EMP_COPY + EMP_AUDIT + trigger
CREATE TABLE emp_copy AS
SELECT * FROM employees;

CREATE TABLE emp_audit (
    audit_id    NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    emp_id      NUMBER NOT NULL,
    old_salary  NUMBER NOT NULL,
    new_salary  NUMBER NOT NULL,
    change_date TIMESTAMP DEFAULT SYSTIMESTAMP,
    changed_by  VARCHAR2(50)
);

CREATE OR REPLACE TRIGGER trg_salary_increase
AFTER UPDATE OF salary ON emp_copy
FOR EACH ROW
WHEN (NEW.salary > OLD.salary)
BEGIN
    INSERT INTO emp_audit (
        emp_id,
        old_salary,
        new_salary,
        change_date,
        changed_by
```

```

    )
    VALUES (
        :OLD.employee_id,
        :OLD.salary,
        :NEW.salary,
        SYSTIMESTAMP,
        USER
    );
END;
/
SHOW ERRORS;

-- D. TEST RUNS + EXPECTED OUTPUT SHAPE

PROMPT ===== Test Q1: raise_amount on EMPLOYEES (employee_id=100) =====

-- Before
SELECT employee_id, first_name, job_id, salary
FROM employees
WHERE employee_id = 100;

-- Raise 10%
EXEC raise_amount(100, 0.10);

-- After
SELECT employee_id, first_name, job_id, salary
FROM employees
WHERE employee_id = 100;

PROMPT ===== Test Q2: DisplayEmployeeInfo(100) =====
EXEC DisplayEmployeeInfo(100);

PROMPT ===== Test Q3: insert_department(300, 'Computer Science') =====
BEGIN
    insert_department(300, 'Computer Science');
END;
/

SELECT department_id, department_name
FROM departments
WHERE department_id = 300;

PROMPT ===== Trigger Ex1: purchase_trig =====

-- Insert before/after cost
INSERT INTO purchase (product, quantity, price)
VALUES ('Bolts', 4, 9.00);

```

```

SELECT product, quantity, price, cost
FROM purchase
ORDER BY product;

INSERT INTO purchase (product, quantity, price)
VALUES ('Nails', 5, 15.50);

SELECT product, quantity, price, cost
FROM purchase
ORDER BY product;

-- Update bolts price, cost auto-updates
UPDATE purchase
SET price = 19.85
WHERE product LIKE 'Bolts%';

SELECT product, quantity, price, cost
FROM purchase
ORDER BY product;

```

- output

	PRODUCT	QUANTITY	PRICE	COST
1	Bolts	4	9	36

	PRODUCT	QUANTITY	PRICE	COST
1	Bolts	4	9	36
2	Nails	5	15.5	77.5

	PRODUCT	QUANTITY	PRICE	COST
1	Bolts	4	19.85	79.4
2	Nails	5	15.5	77.5

exercise 2

- code

```
UPDATE emp_copy
SET salary = salary * 1.20
WHERE employee_id = 143;

COMMIT;

SELECT *
FROM emp_audit;
ORDER BY change_date DESC;
```

- output

AUDIT_ID	EMP_ID	OLD_SALARY	NEW_SALARY	CHANGE_DATE	CHANGED_BY
1	143	2600	3120	22/12/25 11:33:14.543025000 上午	A227154